

从“能跑”到“可维护”的编译器流水线改造

Pascal-S → C Source-to-Source Compiler

课程：编译原理与技术课程设计

汇报时间：2026 年 4 月 15 日

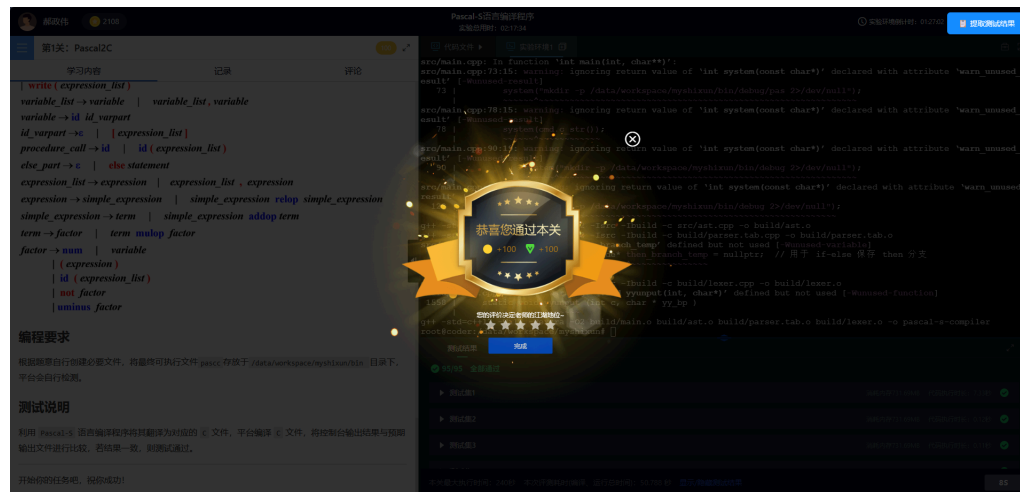
组长汇报 + 模块负责人补充

当前阶段成果：平台测试 **95/95** 全通过，已形成可继续扩展的语法分析 → 语义分析 → 代码生成流水线。

1. 项目目标与当前进展

- 目标：实现一个 Pascal-S 到 C 的源到源编译器，支持基本语法、控制流、函数、数组、错误报告，并在平台测试中正确翻译与运行。
- 阶段性任务：完成总体设计与详细设计，打通完整编译流程，并在测试平台上稳定通过公开与隐藏用例。
- 当前状态：编译器主流程、语义检查、记录类型扩展、自建测试集和平台构建链都已完成。
- 核心结果：Educoder 平台最终评测 **95/95**，本地自建课程测试与严格 C11 检查也已通过。

本阶段重点不是“再堆功能”，而是把编译器升级成更清晰、可维护、可扩展的工程结构。



平台阶段性结果：Educoder 评测 **95/95** 全通过

2. 总体架构设计

输入源码	词法 / 语法分析	AST	语义分析	C 代码生成
.pas 文件	lexer.l + parser.y	ast.*	semantic_analyzer.* + symbol_table.* + type_resolver.*	codegen.* 输出 .c

- 旧结构的问题：解析阶段混入了符号表操作和类型判断，导致 parser、副作用、代码生成耦合严重。
- 新结构的改进：parser 只负责建 AST；semantic analyzer 独立遍历 AST；codegen 基于语义结果输出 C。
- 收益：模块边界更清晰，功能扩展时不会把逻辑写散在多个阶段里。

3. 前端设计：词法分析、语法分析、AST

词法分析

- 基于 flex, 识别关键字、标识符、常量、运算符和注释。
- 维护行列号, 为后续 syntax / semantic error 的 caret diagnostics 提供定位信息。
- 扩展了布尔字面量和与 record 相关的词法入口。

语法分析与 AST

- 基于 bison 按 Pascal-S 文法构建 AST。
- 引入 ParserState 管理解析期上下文, 减少静态全局状态。
- AST 统一承载声明、表达式、语句、函数调用、数组访问、record 字段访问等结构。

这一层的目标是“把程序结构解析正确”, 而不是“顺便做类型检查”。

4. 词法分析实现要点

- flex 根据正则规则自动生成词法分析器，开发时只需描述“什么字符模式对应什么 token”。
- 关键字采用大小写不敏感匹配，例如 program、const、var 都能被识别。
- 标识符统一转为小写存储，符合 Pascal-S 大小写不敏感的语言特性。
- 词法阶段同时维护 line / column，为后续 caret diagnostics 提供定位基础。

输入模式	输出 token
program	PROGRAM
abc123	IDENTIFIER
123	INTEGER_LITERAL
true / false	BOOLEAN_LITERAL
:=	ASSIGN

讲解主线：字符流先被切分成 token 流，后面的 parser 才能按文法进行归约。

5. 中端设计：语义分析与类型系统

- 新增 SemanticAnalyzer, 独立完成变量、常量、函数、参数、作用域和类型相关检查。
- 通过 SymbolTable 管理符号插入与查找, 支持作用域入栈 / 出栈。
- 通过 TypeResolver 统一表达式类型推断、record 字段解析、数组元素类型查询, 避免 semantic 和 codegen 各写一套逻辑。
- 已支持的典型语义检查: 未声明标识符、赋值类型不匹配、参数错误、引用参数左值约束、record 字段不存在等。

关键改进: **类型知识只维护一份**, 避免语义分析与代码生成出现类型规则漂移。

6. 语义分析实现要点

- SymbolTable 采用作用域栈结构：进入函数时入栈，离开函数时出栈。
- SemanticAnalyzer 遍历 AST，对变量使用、函数调用、赋值语句和条件表达式进行合法性检查。
- TypeResolver 统一处理表达式类型、数组元素类型和 record 字段类型，避免多个模块重复推断。
- 典型错误包括：未声明标识符、参数数量或类型错误、record 字段不存在、引用参数不是左值。

阶段	作用
符号表	回答“这个名字是什么”
作用域	回答“当前位置能不能访问它”
类型检查	回答“这条语句在语义上是否合法”
TypeResolver	回答“这个表达式最终是什么类型”

讲解主线：语法正确不代表程序正确，语义分析负责判断这棵 AST 在“含义上”是否成立。

7. 后端设计：代码生成与语言扩展

代码生成

- 输出 C 目标代码，支持声明、表达式、控制流、函数、数组、过程调用。
- 统一调用参数 lowering 与声明输出，减少重复代码。
- 尽量生成更标准的 C，避免依赖 GNU 特有扩展。

本阶段新增支持

- record 结构定义与多级字段访问。
- `program x(input, output);` 头部。
- Pascal 数组任意下界到 C 下标偏移映射。
- 函数调用作语句、布尔字面量与更完善的表达式翻译。

后端目标不只是“把代码拼出来”，而是把 Pascal-S 语义稳定地映射为 C。

8. 代码生成实现要点

- 代码生成阶段遍历 AST，把声明、语句、表达式和函数翻译成等价的 C 代码。
- Pascal 数组允许任意下界，因此生成数组访问时要做下标偏移；record 则映射为 C 的 struct。
- var 参数会被翻译成指针参数，函数和过程调用还要处理参数求值顺序与临时变量。
- 本阶段对调用 lowering 与声明输出做了去重，使代码生成逻辑更集中、更容易维护。

```
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> .\build-win64\pascc.exe .\course_tests\success\record_nested_access.pas -o .\docs\presentation\assets\record_nested_access_demo.c
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> Get-Content .\course_tests\success\record_nested_access.pas
program RecordNestedAccess(input, output);
var
  point : record
    x : integer;
    inner : record
      y : integer;
    end;
  end;
begin
  point.x := 3;
  point.inner.y := point.x + 4;
  write(point.inner.y);
end.
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> Get-Content .\docs\presentation\assets\record_nested_access_demo.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <stdbool.h>

struct pas_record_1 {
    int y;
};

struct pas_record_2 {
    int x;
    struct pas_record_1 inner;
};

struct pas_record_2 point;

int main() {
    point.x = 3;
    point.inner.y = point.x + 4;
    printf("%d", point.inner.y);
    return 0;
}
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> |
```

record 样例翻译结果：Pascal-S 结构被映射成 C struct

9. 错误诊断

- 错误提示从简单行号升级为 caret diagnostics：显示源码原文，并在出错位置下方标注定位箭头。
- 诊断覆盖 syntax error 和 semantic error，便于调试和课堂展示。
- 这一能力依赖词法阶段的行列号记录、主流程中的源码缓存，以及错误模块中的统一输出逻辑。
- 相比只报 “Error + 行号” 的方式，这种形式更接近现代编译器，也更适合课堂展示。

```
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler> .\build-mingw2\pascc.exe .\course_tests\errors\semantic_rec
ord_unknown_field.pas
[Semantic Error] line 7, col 9: Unknown record field: missing
      node.missing := 1;
               ^
PS D:\pascal-s-compiler\pascal-s-compiler\pascal-s-compiler>
```

语义错误示例：record 字段不存在时的 caret diagnostics

10. 测试验证

- 除平台测试外，新增 `course_tests/` 课程自建测试集，覆盖成功用例与错误用例。
- 增加严格生成代码检查，以更严格的 C11 方式验证输出 `.c` 的规范性。
- 验证目标不是只看通过率，还要证明生成结果可编译、可运行、可定位问题。

验证维度	结果	说明
Educoder 平台	95/95	公开 + 隐藏用例全部通过
课程自建测试	13/13	成功与失败场景均覆盖
严格 C11 检查	8/8	验证生成 C 的规范性

11. 项目亮点与加分项

- 在基础 Pascal-S 功能之外，补充支持了 record 结构与多级字段访问，使语言子集更完整。
- 将错误提示升级为带源码行和定位标记的诊断形式，改善了可调试性与展示效果。
- 围绕平台数据之外补充课程自建测试集，使功能验证从“能过样例”扩展到“能自证正确”。
- 将语义分析从 parser 中拆分出来，形成更清晰的语法分析 → 语义分析 → 代码生成流水线。
- 在输出侧增加更严格的 C11 检查，使项目不仅关注通过率，也关注生成结果的规范性。

这些改动让项目从“完成基本翻译”提升到“具备扩展性、诊断能力与自我验证能力”的阶段。

12. 当前问题与下一步计划

- 当前仍可继续优化的方向：进一步拆分 codegen 大文件、继续瘦身 parser.y、补充更多教学型文档。
- 下一步工作：补齐最终报告材料、整理模块说明、继续沉淀自建测试集，保证后续演示与验收稳定。
- 最终交付目标：不仅“平台通过”，还要让代码结构、测试设计和文档表达都能支撑课程设计答辩。

阶段结论：功能已达标，接下来重点转向文档表达与展示质量。